COSC 2637/2633 Big Data Processing
Assignment 2
Advanced Winter Olympic Medal Analysis Using Pig Latin

Assessment Type	– This assignment may be completed **individually** or in **groups of two or three** students. – Submit online via Canvas → Assignment 2. – Marks awarded for meeting requirements as closely as possible. – Clarifications/updates may be made via announcements or relevant discussion forums.
Due Date	Due at 23:59, 27/09/2026, Friday
Marks	25

Overview

In this assignment, you will use Apache Pig on the RMIT-provided AWS EMR cluster to process large datasets in HDFS. You'll practise loading data, joining tables, aggregating metrics, and extending Pig with Jython UDF.

Learning Outcomes

The key course learning outcomes are:
- CLO 1: Model and implement efficient big data solutions for various application areas using appropriately selected algorithms and data structures.
- CLO 2: Analyse methods and algorithms, to compare and evaluate them with respect to time and space requirements and make appropriate design choices when solving real-world problems.
- CLO 3: Motivate and explain trade-offs in big data processing technique design and analysis in written and oral form.
- CLO 4: Explain the Big Data Fundamentals, including the evolution of Big Data, the characteristics of Big Data and the challenges introduced.
- CLO 6: Apply the novel architectures and platforms introduced for big data.

Group Assessment

You may complete this assignment either individually or in a group of two or three students. No other group sizes are permitted. Groups must consist of students who are enrolled at the same level (either all Undergraduate COSC2633 students or all Postgraduate COSC2637 students). All members are expected to contribute equally. Unless a contribution dispute is reported (and supported by the work log), all group members will receive the same grade. Individual work is allowed but not encouraged. Please note that no bonus marks will be awarded for completing the work individually.

Group Formation and Locking Policy

- Navigate to **People** in Canvas, then select the **Groups** tab. **Scroll past the A1 groups until you reach the A2 group section.** Choose an empty A2 group and join it together with your group members. Please do not use your A1 group for this assessment. You may keep the same team members, but you must join a A2 group.
- **You are not permitted to create new groups. You are not permitted to rename the groups.** This restriction is in place because student-created groups are not linked to the grading system, which means they cannot be identified by the system during marking.
- Canvas groups are intended only for students completing the assignment in groups of two or three. Students completing the assignment individually do not need to join a Canvas group.
- Groups will be locked 72 hours before the deadline. Once groups are locked, students can no longer join, leave, or modify their groups. This ensures consistency in grading and prevents last-minute changes that could affect group accountability.

Before submitting A2, you must:
1. Ensure your group membership is correct.
2. Verify that all group members are listed in the Canvas A2 group.
3. Submit the assignment only after groups have been finalised and locked.

Please DO NOT submit your assignment before your group has been correctly formed and locked.
You are responsible for forming your group before beginning the assessment. Students who do not correctly form their group before submission will receive a 10% deduction (2.5 marks).

Group Leadership

The first student to join a group on Canvas is assigned as the group leader by default. However, this role can be changed later within the group settings if needed. The group leader is responsible for ensuring that the group is correctly finalised before the deadline. This includes:

- Confirming all required files are prepared and correctly named.
- Verifying the correct members are listed in the group.
- Submitting the final work before the deadline.
- Keeping and submitting the group work log as part of the submission.

The group leader is responsible for coordinating communication and organising weekly meetings. The role does not provide additional authority over other group members, nor does it make the group leader responsible for completing the assignment on behalf of the group or for the actions of other members. All group members share equal responsibility for contributing to the assignment, reviewing the final submission, and complying with the course's academic integrity requirements.

Extension Policy for Group Assignments

Any extension requests for group assignments must be approved by the Course Coordinator. Group extensions will only be considered under exceptional circumstances (**if more than 50% of the group members are unable to contribute at the same time**) and require supporting documentation.

Please note that extensions are granted only for **unexpected circumstances outside a student's control**, as outlined in RMIT policy. Regular commitments such as part-time work or general workload are considered foreseeable and therefore do not typically meet the criteria. Students are expected to plan their work and make every effort to meet the original deadline. The following reasons are **generally not considered valid** for granting an extension, as they are foreseeable or within a student's control:

- **Academic workload & time management:** multiple assignments due in the same week; general study pressure or heavy workload; poor time management or starting late; and/or underestimating the difficulty of the assignment.
- **Employment commitments:** regular part-time or casual work shifts; busy work schedule or increased workload, and/or choosing to take additional shifts.
- **Travel & personal plan:** pre-booked travel, holidays, or social commitments; events arranged in advance; and/or planned relocation or return home arrangements.
- **Group-related issues:** difficulty coordinating with group members; miscommunication within the group; one member being unavailable or unresponsive; and/or late group formation.
- **Technical & submission issues (avoidable):** uploading incorrect files; last-minute technical problems due to late submission; Internet issues at the time of submission (if preventable); and/or failure to check submission requirements.
- **Minor or foreseeable circumstances:** feeling stressed or overwhelmed (without supporting evidence); general tiredness or lack of preparation, and/or known commitments that were planned in advance.

If you wish to apply for an **individual extension**, you must withdraw from your group (if applicable) before the assignment deadline. You will then be required to complete and submit the entire assignment individually. If your individual application is approved, alternative tasks will be given only to you — not to your former group members.

What to Do If a Team Member Isn't Contributing

- Try to resolve the issue first through communication within the group. Document communication attempts (e.g., emails, messages). Keep a simple log of who completed which parts of the work.
- If the situation doesn't improve, you may choose to leave the group, join other groups OR complete the assignment individually.

Please notify the tutor and the course coordinator early if you decide to do so. Adjustments due to non-contributing members will not be granted unless clear evidence is provided.

Late Penalty

Assignments received late and without prior extension approval or special consideration will be penalised by a deduction of 10% of the total score possible per calendar day late for that assessment.

Assessment Details

All inputs are on HDFS, **comma-separated**, and all filenames are **case-sensitive**. Directory and file names are case-sensitive.

- `/countries.csv`: Country code, country name, and region. RUS is retained as a teaching simplification for Russia/OAR/ROC records before 2026.
- `/games.csv`: Winter Olympic host city and host country code from 2006 to 2026.
- `/medal_table.csv`: Medal table records for selected countries. RUS is omitted in 2026.
- `/population.csv`: Approximate population values by year and country for efficiency calculations.

Before You Start

Before implementing any Pig Latin scripts, it is strongly recommended that you spend some time exploring the datasets and understanding how they are related. A good understanding of the data model will make it much easier to design correct Pig workflows and avoid unnecessary complexity.

Consider the following questions:

1. **Explore the structure of each dataset**
   Review the schema and sample records of each csv file (See **Appendix A** for samples.) For each dataset, identify what each record represents; the purpose of each attribute; whether the dataset contains historical, demographic, or performance information.
2. **Identify relationships between tables**
   Determine how the datasets are connected. For example, `medal_table.csv` and `population.csv` can be linked using a combination of `year` and `country_code`.
   Think carefully about which datasets should be joined together; whether a single key or composite key is required; and what information becomes available after each join.
3. **Consider what questions can be answered**
   Before writing any code, consider what insights can be generated from the available data. Examples include: how did medal performance change over time? which country was most efficient relative to its population?
   Students who spend time understanding the datasets before coding typically produce simpler, more accurate, and easier-to-debug solutions.
4. **Think About the Data Flow and Processing Strategy**
   Think carefully about how the data will move through your solution. Consider: which input files are required for the task; which datasets need to be joined together; whether filtering should be applied before or after a join; whether grouping and aggregation are required; how the final results should be sorted or ranked; and what/how information should appear in the output.
   You should also consider the efficiency of your solution, not just its correctness. Different implementations may produce the same result but require very different amounts of processing.
   A well-designed solution should not only produce the correct answer but should also demonstrate an understanding of efficient data processing principles.
5. **Validate Your Results**
   Do not assume your script is correct simply because it executes successfully. You should:
   - Manually verify a small number of records.
   - Check intermediate outputs.
   - Confirm that joins produce the expected number of records.
   - Verify that rankings, averages, and totals are reasonable.
   - Compare your results with hand calculations where possible.

Task 1 – Regional Ranking by Country Medal Efficiency

(Task for Undergraduate COSC2633 Students)

While traditional Olympic medal tables rank countries by the total number of medals won, such rankings often favor countries with large populations. A country with hundreds of millions of people may naturally have a larger pool of athletes than a country with only a few million residents.

To gain a different perspective, sports analysts sometimes examine medal efficiency, which measures Olympic success relative to population size. This allows countries of different sizes to be compared more fairly and helps identify nations that achieve exceptional sporting performance despite having relatively small populations. For example, a country winning 20 medals with a population of 5 million may be considered more efficient than a country winning 50 medals with a population of 300 million. By analysing medals on a per-capita basis, we can better understand which countries are most effective at converting population resources into Olympic success.

Pig Latin Requirement (17 marks)

You are required to rank countries within each region according to their medal efficiency for each Olympic year.

1. For each country in each Olympic year, calculate:
   ```
   total_medals = gold + silver + bronze
   medals_per_million = (total_medals / population) * 1,000,000
   ```
2. Then, for each combination of `year, region`, rank countries from highest to lowest `medals_per_million`.
3. Ranking rules:
   i. Higher `medals_per_million` ranks higher.
   ii. If tied, higher `total_medals` ranks higher.
   iii. If still tied, higher `number of gold medals` ranks higher.
   iv. If still tied, sort by `country_code` in ascending order.
4. Name your Pig scripts as `task1.pig`.

Jython UDF Requirement (5 marks)

This is an advanced research task. You are required to implement a **Jython User Defined Function (UDF)** to perform the ranking within each (`year, region`) group. **Name your Jython UDF file as `task1.py`.**

The purpose of this requirement is to demonstrate how Pig Latin can be extended using custom UDFs when built-in operators are insufficient or when custom processing logic is required. While Pig provides powerful data transformation capabilities, ranking records within groups is often more naturally expressed using procedural logic. By implementing a Jython UDF, you will gain experience integrating Pig Latin with external code and applying custom algorithms to grouped data. This is a common practice in large-scale data processing systems where domain-specific calculations, classifications, or ranking rules cannot be easily implemented using built-in operators alone. In this task, the UDF is responsible for sorting countries within each (`year, region`) group according to the specified ranking rules and assigning the corresponding rank numbers. This provides an opportunity to demonstrate both Pig Latin data processing skills and the ability to extend Pig using Jython.

If you are unfamiliar with Pig UDFs, you should review the official Apache Pig documentation before starting your implementation.

Official Apache Pig UDF Documentation can be found at Apache Pig UDF Guide (Version 0.18.0).

The documentation explains:

- What a User Defined Function (UDF) is.
- How to register a Jython UDF in a Pig script.
- How to pass parameters from Pig Latin to a Python function.
- How to define input and output schemas.
- How to return values from a UDF.
- Examples of custom processing logic implemented using Jython.

Students are encouraged to experiment with simple UDFs before attempting the assignment tasks. For example, create a UDF that classifies values, formats output, or performs a custom calculation, and then gradually extend it to implement the required ranking logic. The official documentation provides examples and usage patterns that may be useful when developing your solution.

Region Parameter Requirement (3 marks)

To make the analysis more flexible, your Pig script must accept a **region** as a command-line parameter. The script should only process and rank countries belonging to the specified region. For example:

```
pig -x mapreduce -param REGION=Europe task1.pig
```

The region must not be hard coded in the Pig script. Only countries belonging to the specified region should appear in the final output. This requirement allows the same Pig script to be reused for different regions without modifying the source code and reflects a common practice in data engineering, where scripts are parameterised to support different inputs and execution scenarios.

For information about using parameters in Pig scripts, refer to:
https://pig.apache.org/docs/r0.18.0/cont.html#Parameter-Sub

Output Format

```
year,region,rank_no,country_code,country_name,gold,total_medals,population,medals_per_million
```

The final output must be sorted by `year ASC, rank_no ASC`.

Example output:

```
2006,Europe,1,NOR,Norway,2,19,4600000,4.13
2006,Europe,2,SUI,Switzerland,5,14,7500000,1.87
2006,Europe,3,SWE,Sweden,7,14,9100000,1.54
2006,Europe,4,GER,Germany,11,29,82400000,0.35
2006,Europe,5,ITA,Italy,5,11,58600000,0.19
2006,Europe,6,RUS,Russia / ROC / OAR,8,22,143200000,0.15
2010,Europe,1,NOR,Norway,9,23,4900000,4.69
2010,Europe,2,SWE,Sweden,5,11,9400000,1.17
...
...
2026,Europe,1,NOR,Norway,18,41,5700000,7.19
2026,Europe,2,SUI,Switzerland,6,23,8900000,2.58
2026,Europe,3,SWE,Sweden,8,18,10600000,1.7
2026,Europe,4,ITA,Italy,10,30,59200000,0.51
2026,Europe,5,GER,Germany,8,26,84000000,0.31
```

HDFS Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Output/task1

Directory and file names are case-sensitive. You must use names like "task1" and not "Task1" or "TASK1". Failure to follow this naming convention may result in execution and grading issues.

Task 2 – Gold vs Total Medals: Winter Olympics Ranking Analysis

(Task for Postgraduate COSC2637 Students)

The Winter Olympic Games are one of the world's largest international sporting events, attracting athletes from countries across the globe. After each Games, media organisations, sports analysts, and national Olympic committees closely examine the medal table to evaluate the performance of participating countries.

Different stakeholders often view Olympic success from different perspectives. Some focus on **gold medals**, which emphasise championship performances and the ability to produce Olympic winners. Others focus on **total medals**, which measure the overall depth and consistency of a country's sporting program.

For example, a country may win fewer gold medals but achieve a higher total medal count due to strong performances across many events. As a result, rankings based on gold medals and rankings based on total medals do not always produce the same results.

In this task, you are acting as a sports data analyst preparing a summary report for Olympic officials and media organizations. Your objective is to identify the leading countries at each Winter Olympic Games and compare how the rankings change when different performance measures are used.

Pig Latin Requirement (17 marks)

Using the Winter Olympic datasets, analyze each Olympic Games from 2006 to 2026 and produce a summary that:

- Identifies the `top three` countries ranked by `gold medals`.
- Identifies the `top three` countries ranked by `total medals`.
- Displays the `full country names` rather than country codes.
- Includes the `host city` for each Olympic Games.
- Name your Pig scripts as `task2.pig`.

This analysis helps reveal the difference between countries that dominate through championship performances and countries that demonstrate consistent success across a wide range of events.

Jython UDF Requirement (5 marks)

This is an advanced research task.

You are required to use a **Jython User Defined Function (UDF)** to produce the exact pretty-printed report format. The UDF should handle the ranking and formatting logic that is difficult to express cleanly using Pig Latin alone. **Name your Python UDF file as** `task2.py`**.**

The final output must follow the required format exactly. Hard-coded final outputs are not acceptable.

```
2026 Milan-Cortina
By Gold Medals:
1. Norway (18) | 2. United States (12) | 3. Italy (10)
By Total Medals:
1. Norway (41) | 2. United States (33) | 3. Italy (30)
```

This requirement is included to assess your ability to combine Pig Latin with external procedural logic. In real-world data engineering workflows, systems such as Pig are often used to transform and prepare data, while custom functions or downstream programs are used to format, classify, rank, or enrich the results. Therefore, this task requires you to demonstrate both data-processing skills and the ability to extend Pig with a Jython UDF.

If you are unfamiliar with Pig UDFs, you should review the official Apache Pig documentation before starting your implementation.

Official Apache Pig UDF Documentation can be found at Apache Pig UDF Guide (Version 0.18.0).

The documentation explains:

- What a User Defined Function (UDF) is.
- How to register a Jython UDF in a Pig script.
- How to pass parameters from Pig Latin to a Python function.
- How to define input and output schemas.
- How to return values from a UDF.
- Examples of custom processing logic implemented using Jython.

Students are encouraged to experiment with simple UDFs before attempting the assignment tasks. For example, create a UDF that classifies values, formats output, or performs a custom calculation, and then gradually extend it to implement the required ranking logic. The official documentation provides examples and usage patterns that may be useful when developing your solution.

Region Parameter Requirement (3 marks)

To make the analysis more flexible, your Pig script must accept **START_YEAR** and **END_YEAR** as command-line parameters. These parameters allow the analysis to be performed on a selected range of Olympic Games rather than processing all years in the dataset. This enables users to investigate specific Olympic periods, compare country performance across multiple Games, and analyze trends over time without modifying the Pig script. The year range must not be hard coded in the implementation.

For example:

```
pig -x mapreduce -param START_YEAR=2010 -param END_YEAR=2020 task2.pig
```

Example output:

```
2010 Vancouver
By Gold Medals:
1. Canada (14) | 2. Germany (10) | 3. Norway (9)
By Total Medals:
1. United States (37) | 2. Germany (30) | 3. Canada (26)

2014 Sochi
By Gold Medals:
1. Russia (13) | 2. Norway (11) | 3. Canada (10)
By Total Medals:
1. Russia (33) | 2. United States (28) | 3. Norway (26)

2018 PyeongChang
By Gold Medals:
1. Germany (14) | 2. Norway (14) | 3. Canada (11)
By Total Medals:
1. Norway (39) | 2. Germany (31) | 3. Canada (29)
```

For information about using parameters in Pig scripts, refer to:
https://pig.apache.org/docs/r0.18.0/cont.html#Parameter-Sub

HDFS Output Directory Structure

You must follow the exact naming conventions for all input and output directories and files on HDFS.

- /Output/task2

Directory and file names are case-sensitive. You must use names like "task2" and not "Task2" or "TASK2". Failure to follow this naming convention may result in execution and grading issues.

Submission Instructions

Your assignment should follow the requirement below and submit via Canvas > Assignment 2. Assessment declaration: when you submit work electronically, you agree to the assessment declaration.

Failure to follow the format requirements incurs up to 10 marks penalty.

1. Prepare your zip file

- Place all required files into a single folder, including your pig file (task1.pig OR task2.pig) and your UDF file (task1.py OR task2.py).
- **Do NOT include any subfolders.**
- Compress the folder into a zip file.

2. Prepare a pdf file for Turnitin plagiarism check

- Copy and paste the **pig code** and **Jython UDF code** into a plain text editor.
- Save this as a **pdf file**.
- **The PDF must be uploaded as an individual file, not included inside the ZIP file,** to ensure it is correctly processed by Turnitin for plagiarism checking.

Please note that, in accordance with RMIT policy, we cannot mark a submission that has not been processed through the required plagiarism checking system. Failure to submit the PDF separately will result in your work being unable to be assessed.

3. Prepare your worklog doc file

- Please double-check that you have completed all two sections.
- Use the **Submission Declaration Checklist** to verify that your submission satisfies all submission requirements before submitting.

Name your zip file and pdf file using the following format:

`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.zip`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.pdf`
`<GroupLeaderStudentID>_Group<GroupNumber>_<UG/PG>.doc`

Replace:

- `GroupLeaderStudentID` with the student number of the group leader
- `GroupNumber` with your group number (e.g., 3)
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example: `s1234567_Group3_UG.zip` and `s1234567_Group3_UG.pdf`
The group leader is responsible for submitting the group's final zip file. Duplicate submissions from the same group will not be accepted.

If you are completing A1 individually, name your zip file using the following format:

`<StudentID>_<UG/PG>.zip`
`<StudentID>_<UG/PG>.pdf`
`<StudentID>_<UG/PG>.doc`

Replace:

- `StudentID` with your actual student number
- `PG` if you are in a postgraduate program; `UG` if you are in an undergraduate program

Example**:** `s1234567_UG.zip` and `s1234567_UG.pdf`

Both group submissions and individual submissions are required to include a work log. For group submissions, the work log must include a member contribution section (Section 1) that clearly describes each member's contributions. For individual submissions, a work log is still required; however, the member contribution section is not applicable and does not need to be completed.

Important

- You must submit 3 files: **the zip file, the doc file, and the pdf file** for Turnitin plagiarism check.
- **Failure to submit the required pdf/doc file separately from the zip file will result in a 10% penalty (2.5 marks).**
- Submissions that do not include the required PDF or Word document will incur a 10% penalty for each missing file (2.5 marks per file).
- Submissions where the PDF **fails the Turnitin plagiarism check**, **will not be graded**.
- Please verify your submission carefully before the deadline. If you submit the **wrong file**, a **corrupted zip file**, or **a submission that does not comply with the required file structure**, any resubmission made after the deadline will be subject to the standard late submission penalty.
- Late submission penalties are applied automatically by the system based on the assessment due date and the time of submission. **The penalty is determined by when you submit your work, not when you complete it.** Therefore, even if you completed the assessment before the due date, a late submission would still incur the applicable late penalty.
- Work must run on the **RMIT-provided EMR** cluster.
- Please note that markers will not modify or adjust submitted code during marking.

Functional Requirements

Failure to follow the requirements incurs up to 5 marks penalty.

– The code must be well written using a good coding style.
– The codes and scripts must come with concise and clear comments to explain the logical flow of the program.

Academic Integrity and Plagiarism (standard warning)

RMIT University treats plagiarism as a very serious offense constituting misconduct.
Plagiarism covers a variety of inappropriate behaviours, including:

– Failure to properly document a source.
– Copyright material from the internet or databases
– Collusion between students

For further information on our policies and procedures, please refer to
https://www.rmit.edu.au/students/student-essentials/rights-and-responsibilities/academic-integrity

Assessment Criteria

Assessment for this Pig Latin assignment will be based on two primary criteria:

1. **The correctness of the program's output**: the program must produce accurate and complete results as specified in the task requirements.
2. **The correctness of the task design and implementation**: your code should demonstrate a sound understanding of **Pig Latin** principles, including
   - Apply filters as early as possible to reduce the amount of data processed in later stages.
   - If only a subset of records is needed, filter the data before performing joins whenever possible.
   - Avoid joining datasets that are not required for the task.
   - Avoid calculating values that are never used in the final output.
   - Consider whether a join should occur before or after aggregation.
   - Think about how grouping operations affect the amount of data being processed.

Submissions in which the code does not run or produces no output will receive no more than 20% of the available marks for this assessment.

Appendix A – Structure and Sample Data for CSV Files

countries.csv

NOR	Norway	Europe
GER	Germany	Europe
CHN	China	Asia
USA	United States	North America
CAN	Canada	North America
SWE	Sweden	Europe
KOR	South Korea	Asia
JPN	Japan	Asia
RUS	Russia / ROC / OAR	Europe
ITA	Italy	Europe
SUI	Switzerland	Europe

games.csv

2006	Turin	ITA
2010	Vancouver	CAN
2014	Sochi	RUS
2018	PyeongChang	KOR
2022	Beijing	CHN
2026	Milan-Cortina	ITA

medal_table.csv

2006	NOR	2	8	9
2006	GER	11	12	6
2006	CHN	2	4	5
2006	USA	9	9	7
2006	CAN	7	10	7
2006	SWE	7	2	5
2006	KOR	6	3	2
2006	JPN	1	0	0
2006	RUS	8	6	8
2006	ITA	5	0	6
2006	SUI	5	4	5
2010	NOR	9	8	6
2010	GER	10	13	7
2010	CHN	5	2	4
2010	USA	9	15	13
2010	CAN	14	7	5
2010	SWE	5	2	4
2010	KOR	6	6	2
2010	JPN	0	3	2
2010	RUS	3	5	7
2010	ITA	1	1	3

population.csv

2006	NOR	4600000
2006	GER	82400000
2006	CHN	1311000000
2006	USA	299000000
2006	CAN	32500000
2006	SWE	9100000
2006	KOR	48400000
2006	JPN	127800000
2006	RUS	143200000
2006	ITA	58600000
2006	SUI	7500000
2010	NOR	4900000
2010	GER	81700000
2010	CHN	1341000000
2010	USA	309000000
2010	CAN	34000000
2010	SWE	9400000
2010	KOR	49500000
2010	JPN	128100000
2010	RUS	142900000
2010	ITA	59300000